ANALISE LÉXICA E TABELA DE SÍMBOLOS

1. Objetivo:

O objetivo deste trabalho é implementar um **analisador léxico** para a linguagem MicroPascal (µ-Pascal), conforme especificado em sala de aula. São regras que devem ser observadas no desenvolvimento deste trabalho prático:

- O trabalho poderá deverá ser desenvolvido em grupos de até 5 alunos;
- Não é permitido o uso de ferramentas para geração do analisador léxico;
- A implementação deverá ser realizada em C;
- Trabalhos total ou parcialmente iguais receberão avaliação nula;
- Ultrapassados cinco (5) dias da data definida para entrega, nenhum trabalho será recebido.

O analisador léxico deverá:

- reconhecer os **tokens da linguagem**
- identificar **erros léxicos**
- manter uma **Tabela de Símbolos (TS)**
- registrar a **posição (linha e coluna)** de cada token
- gerar um **arquivo de saída (.lex)** contendo os tokens reconhecidos

A implementação deverá ser baseada em um **Autômato Finito Determinístico (AFD)**.

2. Funcionamento:

O analisador deverá ler um programa fonte escrito em MicroPascal e, a cada chamada, retornar um token correspondente ao próximo lexema reconhecido. Cada token deverá ser representado por uma estrutura contendo nome do token, lexema, linha e coluna. Espaços em branco, tabulações, quebras de linha e comentários **não são tokens** e devem ser ignorados. Na linguagem MicroPascal, os comentários devem ser colocados entre { e }.

2.1. Tabelas de símbolos

A tabela de símbolos é uma estrutura de dados usada pelo compilador (ou interpretador) para armazenar informações sobre os elementos identificadores do programa, como nomes de variáveis, nomes de funções, nomes de procedimentos, palavras-reservadas (no caso do analisador léxico), constantes e tipos. Ou seja, ela funciona como um “cadastro oficial” dos nomes usados no programa. Neste trabalho prático, uma Tabela de Símbolos (TS) deverá ser utilizada com as seguintes características:

- Inicialmente deve conter todas as palavras reservadas da linguagem;
- Apenas identificadores e palavras reservadas devem ser armazenados na TS;
- Não é permitido inserir o mesmo lexema mais de uma vez; e
- A linguagem é case-insensitive.

Por exemplo, se o analisador encontra:

```
program Exemplo;
var x, y : integer;
```

A tabela pode ficar assim:

Lexema	Tipo de token
program	palavra-reservada
var	palavra-reservada
integer	palavra-reservada
exemplo	identificador
x	identificador
y	identificador

2.2. Erros léxicos

O analisador deve detectar e reportar caractere inválido, string não fechada antes de quebra de linha e comentário não fechado antes do fim de arquivo (caso comentários sejam suportados). Assim, o analisador léxico deve, para cada erro, informar o tipo do erro, a linha e a coluna.

A título de exemplo, para o código

```
program Teste;
var
   x : integer;
begin
   x := 10 @ 2;
end.
```

o analisador léxico deve acusar o caractere inválido **@** na linha **5**.

Em suma, os principais erros léxicos estão relacionados, principalmente, à presença de caracteres inválidos na linguagem MicroPascal, como **@**, **%** e **$**, que não pertencem ao conjunto de símbolos reconhecidos pelo analisador. Além disso, são erros léxicos os referentes a lexemas malformados, como identificadores contendo caracteres não permitidos e possíveis ocorrências de comentários ou strings não suportados pela especificação adotada. Ademais, o analisador léxico deve interromper o reconhecimento normal, registrar o tipo de erro encontrado e informar com precisão a **linha e a coluna** em que o problema ocorreu, permitindo ao usuário localizar e corrigir a inconsistência no programa fonte.

2.3. Tokens de linguagem

Um token é a unidade básica reconhecida pelo analisador léxico de um compilador ou interpretador. Ou seja, representa a classificação de um lexema (sequência de caracteres) segundo as regras da linguagem. Por exemplo, para o código seguinte:

```
x := 10;
```

O analisar léxico deve identificar:

Lexema	Token
x	ID
:=	OP_ASS
10	NUM_INT
;	SMB_SEM

São tokens da linguagem MicroPascal:

1) As seguintes palavras-chave: program, var, integer, real, begin, end, if, then, else, while, do
2) Os identificadores (Sequência iniciada por letra seguida de letras e/ou dígitos) como o token ID
3) Os números inteiros (NUM_INT) e reais com parte decimal obrigatória (NUM_REAL)
4) Os seguintes operadores

   - OP_EQ =
   - OP_NE <>
   - OP_LT <
   - OP_LE <=
   - OP_GT >
   - OP_GE >=
   - OP_AD +
   - OP_MIN -
   - OP_MUL *
   - OP_DIV /
   - OP_ASS :=

5) Os seguintes símbolos:

   - SMB_SEM ;
   - SMB_COM ,
   - SMB_OPA (
   - SMB_CPA)
   - SMB_COL :
   - SMB_DOT .
   - SMB_OBC {
   - SMB_CBC }

6) Outras características da linguagem:

- Palavras-chave são reservadas
- Variáveis devem ser declaradas antes do uso (verificação não é realizada na analise léxica)
- A linguagem não diferencia maiúsculas de minúsculas
- Comentários: conforme definição adotada em sala
- A semântica dos comandos segue o padrão do Pascal

3. Saída do analisador:

O analisador deverá gerar um arquivo **.lex** contendo, para cada token:

<nome_token, lexema> linha coluna

Exemplo:

<KW_PROGRAM, program> 1 1
<ID, exemplo> 1 9
<SMB_SEM, ;> 1 16

Ao final da execução, deve ser gerado em um arquivo **.ts** contendo a Tabela de Símbolos completa e o arquivo **.err** contendo a lista dos erros léxicos encontrados.

4. Entregáveis:

O grupo deverá entregar:

1) Diagrama do AFD (pode ser feito no JFLAP ou, preferencialmente, usando Graphviz);

   - JFLAP: *http://www.jflap.org/*
   - Graphiviz: *https://graphviz.org/download/*

2) Código-fonte completo (todos os códigos e instruções para execução)
3) Relatório técnico contendo:

   - descrição das estruturas de dados
   - descrição das funções implementadas
   - explicação do AFD
   - testes realizados (no mínimo 3 corretos e 3 com erro léxico)
   - saída gerada pelo analisador
   - conteúdo final da Tabela de Símbolos

4.1. Exemplo de programa:

O programa a seguir se chama Exemplo, as variáveis x e y são declaradas como do tipo integer, enquanto z é do tipo real. Além disso, o bloco principal do programa contém:

- Atribuições às variáveis (x, y, z).
- Um comando condicional if que compara x e y.
- Um laço while que multiplica z por 1.5 até que z seja maior que 100, ao mesmo tempo que incrementa x.

```
program Exemplo;

var
   x, y : integer;
   z : real;

begin
   x := 10;
   y := 20;
   z := x + y * 2.5;

   if x > y then
      x := x - 1
   else
      y := y + 1;

   while z <= 100 do
   begin
      z := z * 1.5;
      x := x + 2
   end
end.
```